Name: Rohit Balasaheb Kokare

Class: B.Tech (Information Technology)

Roll No.: 83

PRN No.:UIT23M1083

Assignment No:-03

Week 3 Assignment

Objective

SQL Sales Data Analysis using Subqueries, CTEs, and Window Functions

Analyze the Superstore sales dataset using SQL by applying Subqueries, Common Table Expressions (CTEs), Window Functions, and JOIN operations to solve business-related queries. The assignment focuses on identifying customer sales performance, ranking customers, and generating meaningful business insights.

Tools Used

- MySQL Workbench
- SQL
- Superstore Dataset

Dataset

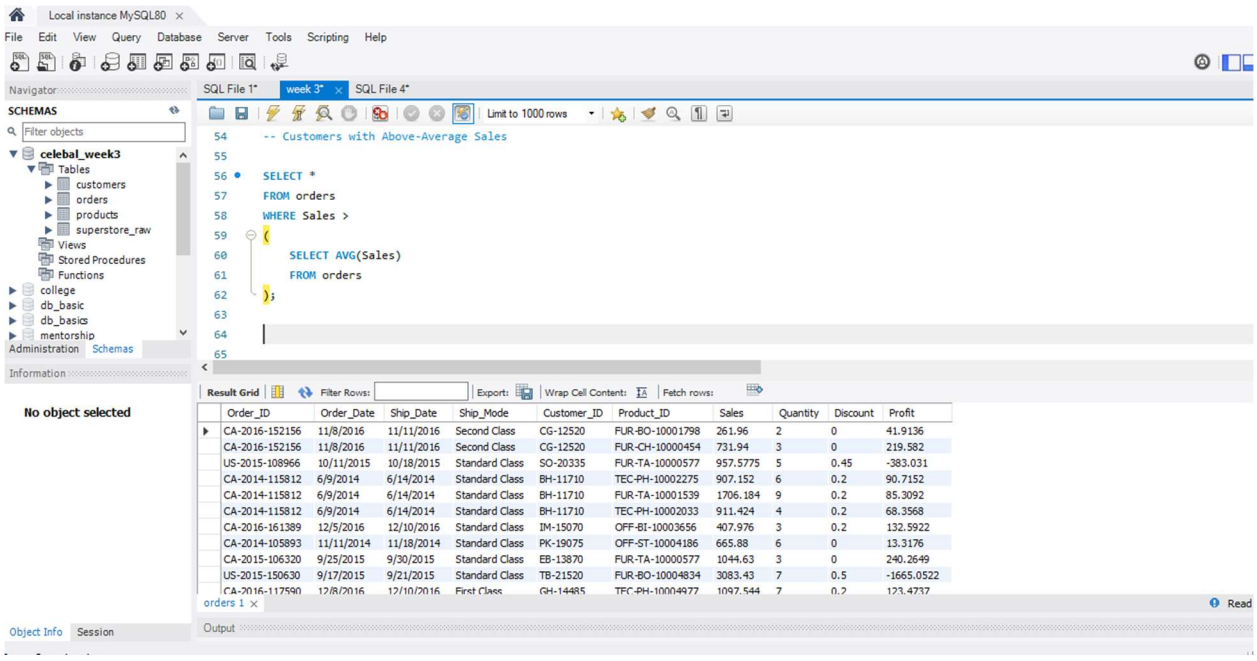
The **Superstore** dataset contains customer, order, and product information, including sales, quantity, profit, category, region, and order details. The dataset was used to create separate tables for customers, orders, and products to perform SQL analysis.

Query 1: Above Average Sales

Query:

```
SELECT *
 FROM orders
WHERE Sales >
(
SELECT AVG(Sales)
FROM orders
);
```

Output:



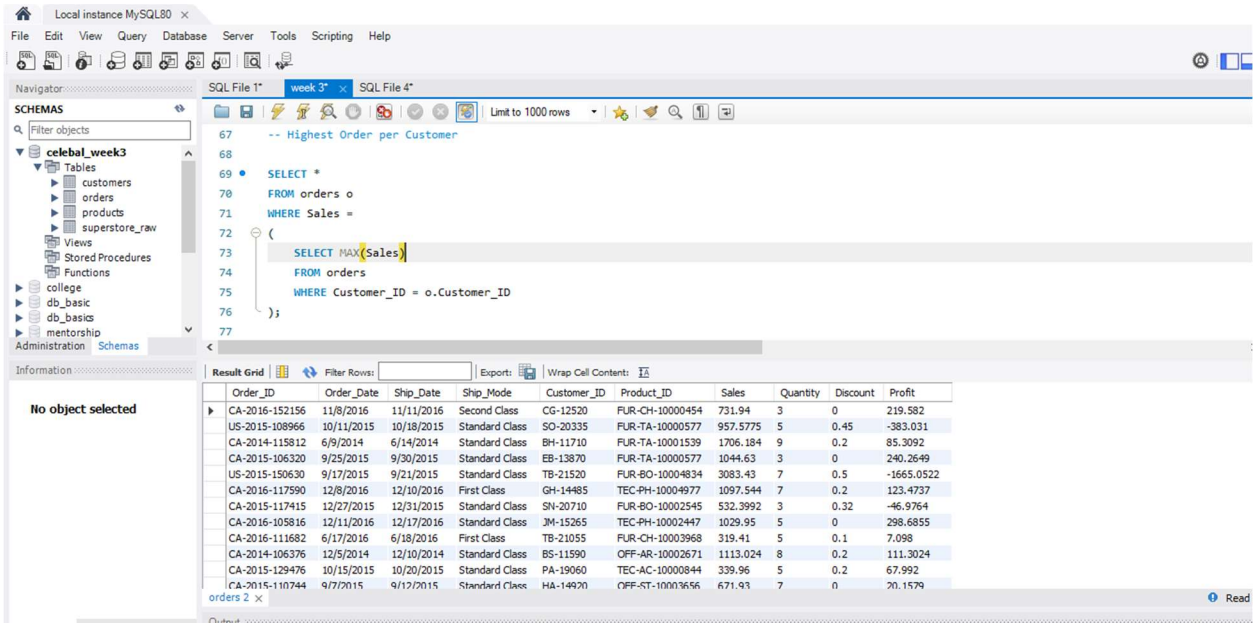
Insight: Displays orders where sales are greater than the average sales.

Query 2: Highest Order per Customer

Query:

```
SELECT *
FROM orders o
WHERE Sales =
(
   SELECT MAX(Sales)
   FROM orders
   WHERE Customer_ID = o.Customer_ID
);
```

Output:



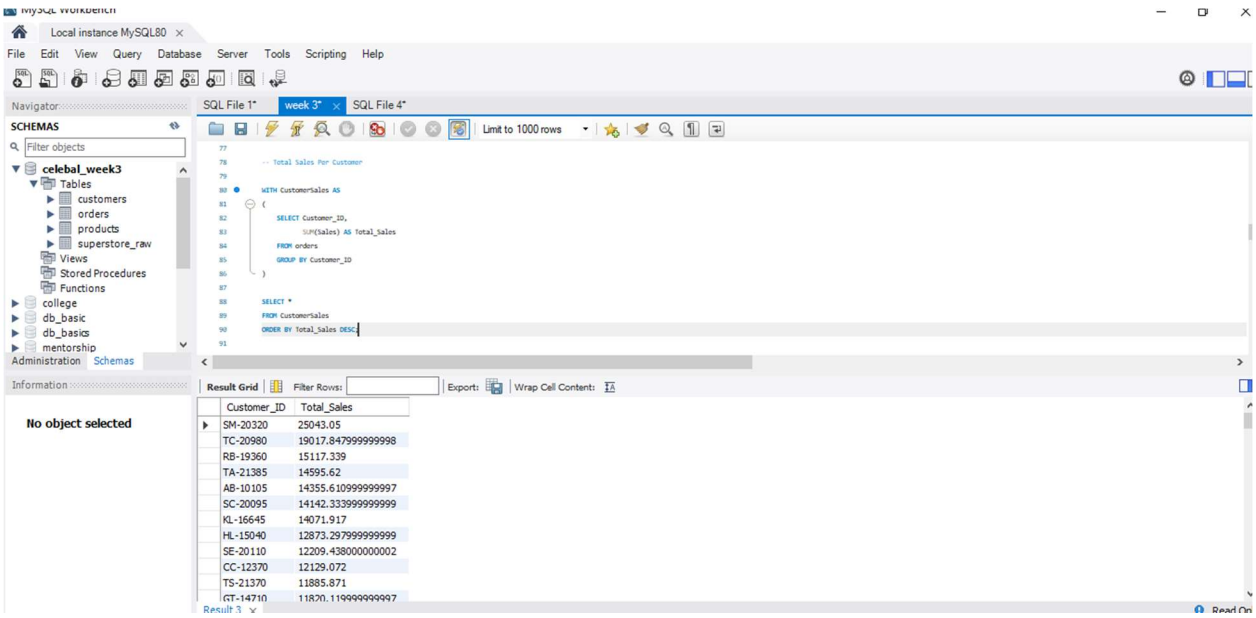
Insight: Identifies the highest-value order placed by each customer.

Query 3: Total Sales per Customer (CTE)

Query:

```
WITH CustomerSales AS
(
   SELECT Customer_ID,
        SUM(Sales) AS Total_Sales
   FROM orders
   GROUP BY Customer_ID
)
SELECT *
FROM CustomerSales
ORDER BY Total_Sales DESC;
```

Output:



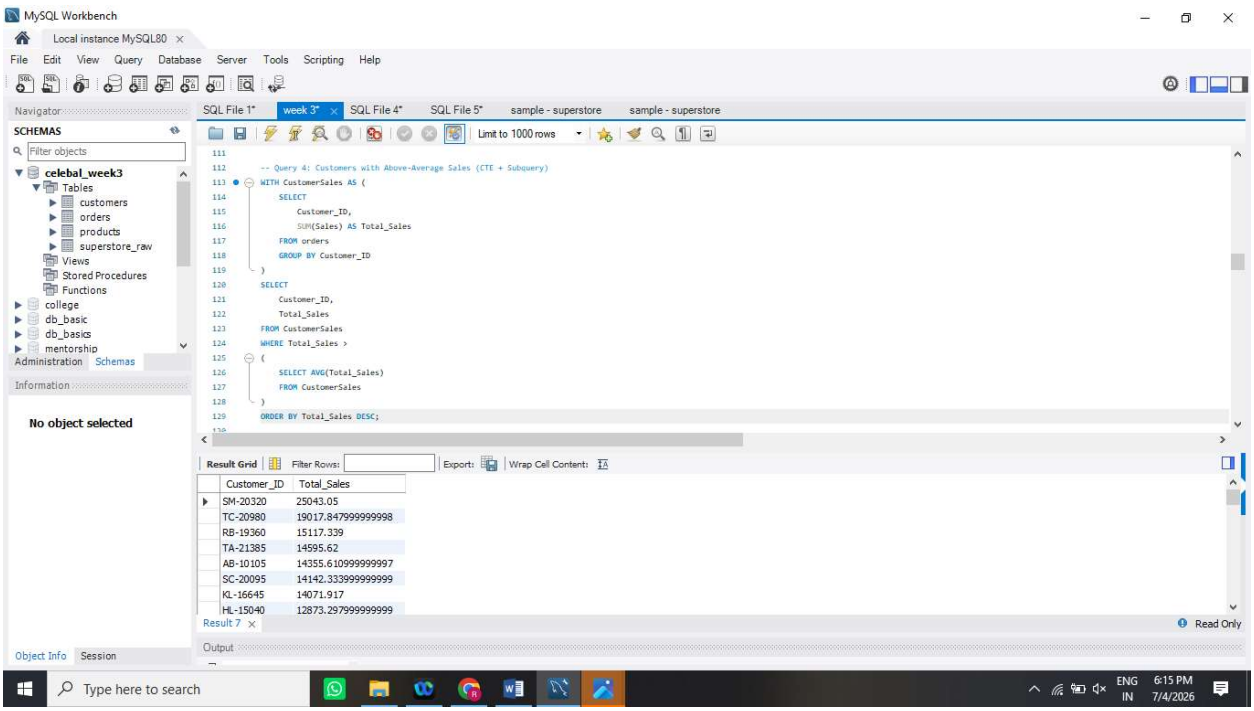
Insight: Calculates total sales for each customer.

Query 4: Customers with Above-Average Sales (CTE + Subquery)

Query:

```
WITH CustomerSales AS (
   SELECT
      Customer_ID,
      SUM(Sales) AS Total_Sales
   FROM orders
   GROUP BY Customer_ID
)
SELECT
   Customer_ID,
   Total_Sales
FROM CustomerSales
WHERE Total_Sales >
(
   SELECT AVG(Total_Sales)
   FROM CustomerSales
)
ORDER BY Total_Sales DESC;
```

Output:



Insight:

This query identifies customers whose total sales are greater than the average total sales, helping identify high-value customers.

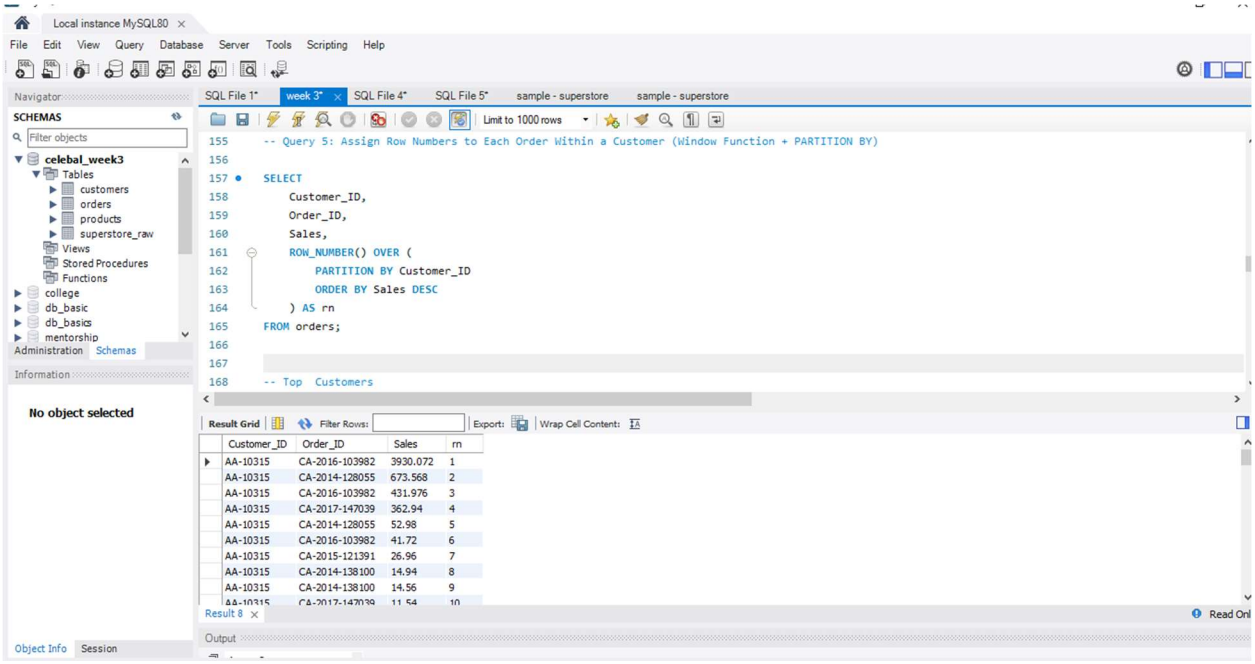
Query 5:Assign Row Numbers to Each Order Within a Customer (Window Function + PARTITION BY)

Query:

```
SELECT
    Customer_ID,
    Order_ID,
    Sales,
    ROW_NUMBER() OVER (
        PARTITION BY Customer_ID
        ORDER BY Sales DESC
```

```
    ) AS Row_Number
FROM orders;
```

Output:



Insight: This query assigns a unique row number to each order within every customer based on sales, making it easier to analyze customer-wise order rankings.

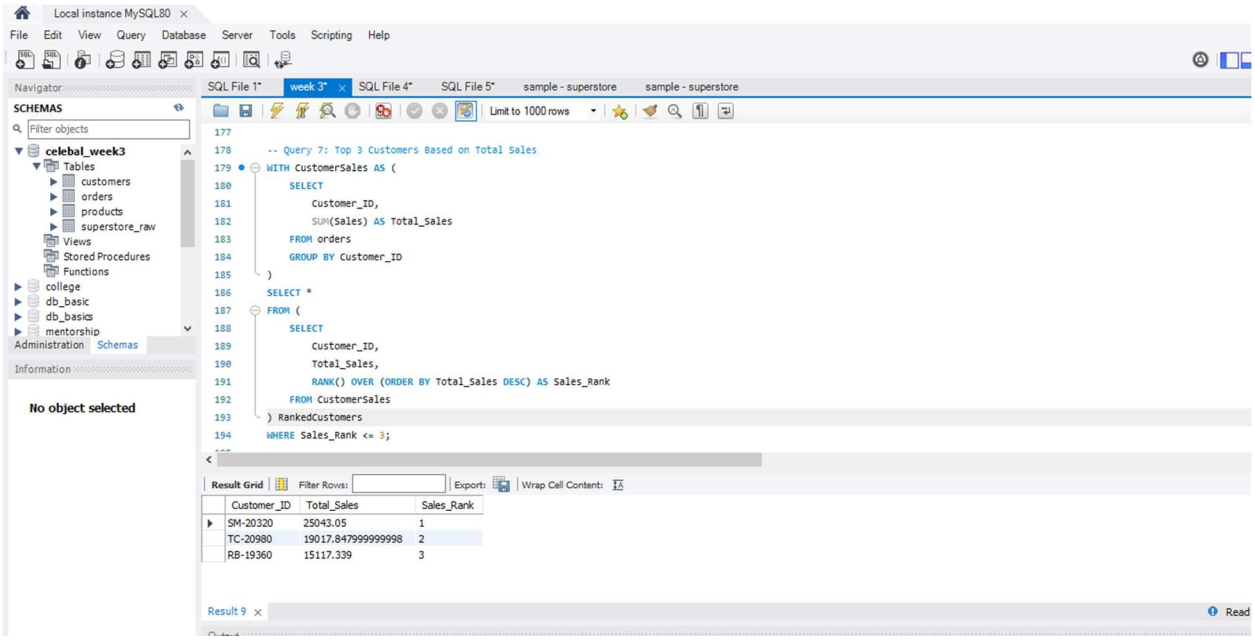
Query 6: Top 3 Customers Based on Total Sales

Query:

```
WITH CustomerSales AS (
    SELECT
        Customer_ID,
        SUM(Sales) AS Total_Sales
    FROM orders
    GROUP BY Customer_ID
)
SELECT *
FROM (
    SELECT
```

```
    Customer_ID,
    Total_Sales,
    RANK() OVER (ORDER BY Total_Sales DESC) AS Sales_Rank
  FROM CustomerSales
) RankedCustomers
WHERE Sales_Rank <= 3;
```

Output:



Insight: This query identifies the top three customers based on total sales using the `RANK()` window function, helping the business recognize its highest-value customers.

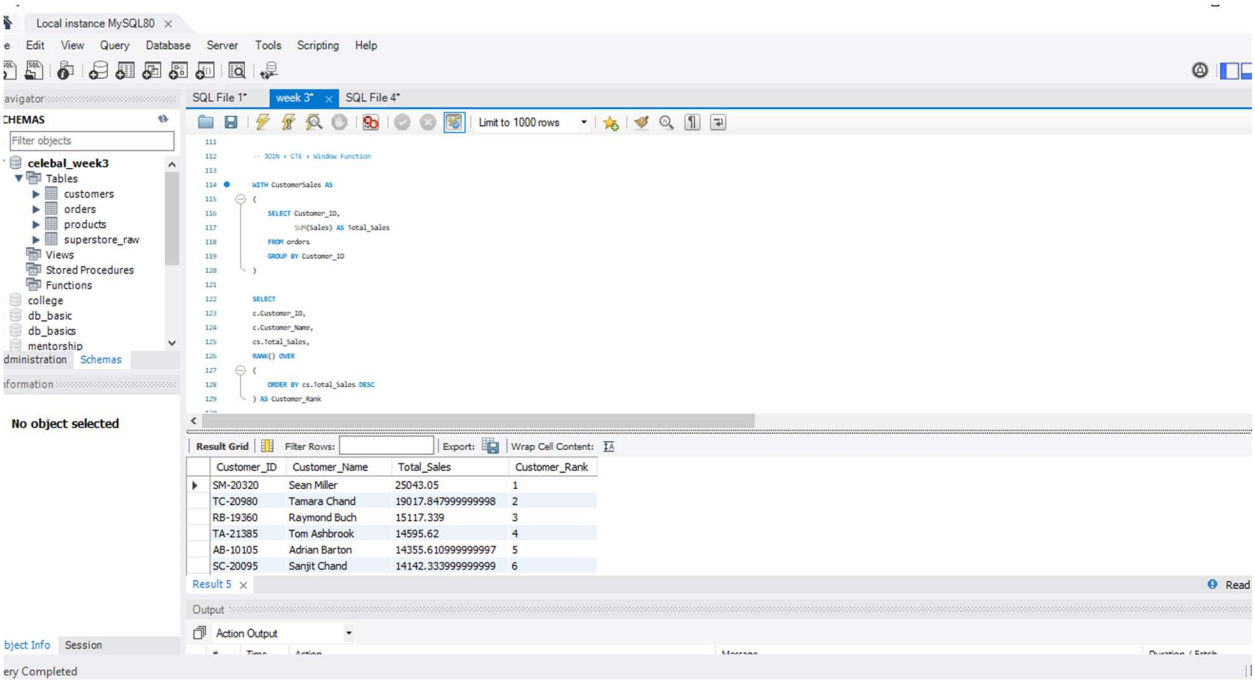
Query 7 :Customer Sales Ranking Report (JOIN + CTE + Window Function)

Query:

```
WITH customer_sales AS (
```

```
    SELECT
        customer_id,
        SUM(sales) AS total_sales
    FROM orders
    GROUP BY customer_id
)
SELECT
    c.customer_id,
    c.customer_name,
    cs.total_sales,
    RANK() OVER (ORDER BY cs.total_sales DESC) AS sales_rank
FROM customers c
JOIN customer_sales cs
ON c.customer_id = cs.customer_id
ORDER BY sales_rank;
```

Output:



Insight: This query combines customer information, total sales, and customer ranking into a single report using JOIN, CTE, and Window Functions, making customer performance analysis easier.

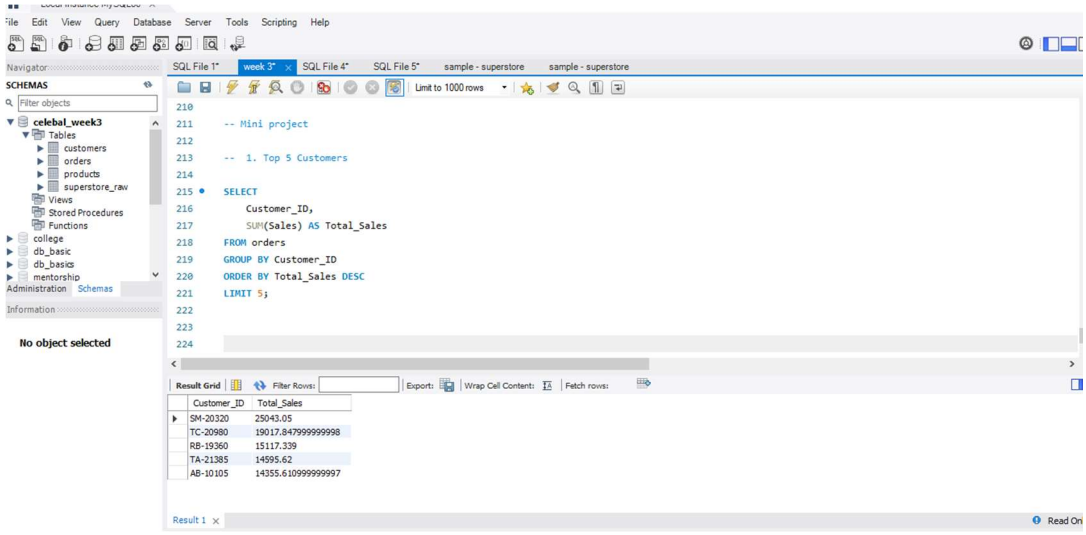
Mini Project: Customer Sales Insights

1. Top 5 Customers

Query:

```
SELECT
Customer_ID,
SUM(Sales) AS Total_Sales
FROM orders
GROUP BY Customer_ID
ORDER BY Total_Sales DESC
LIMIT 5;
```

Output:



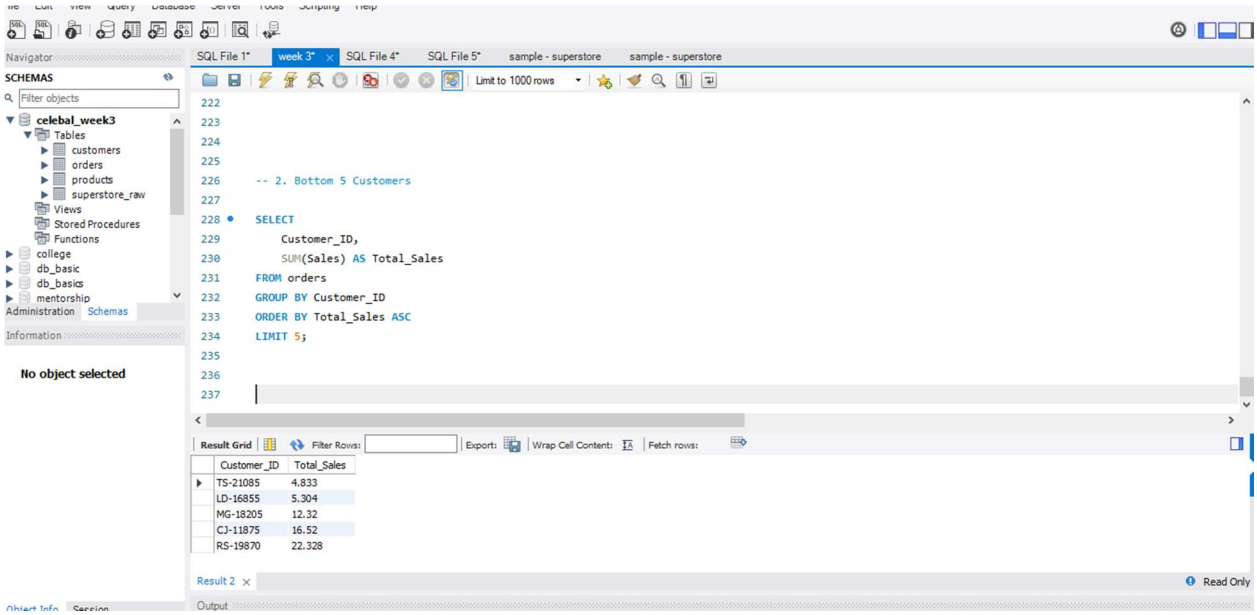
Insight: This query identifies the top five customers based on their total sales, helping the business recognize its highest-value customers.

2. Bottom 5 Customers

Query:

```
SELECT
  Customer_ID,
  SUM(Sales) AS Total_Sales
FROM orders
GROUP BY Customer_ID
ORDER BY Total_Sales ASC
LIMIT 5;
```

Output:



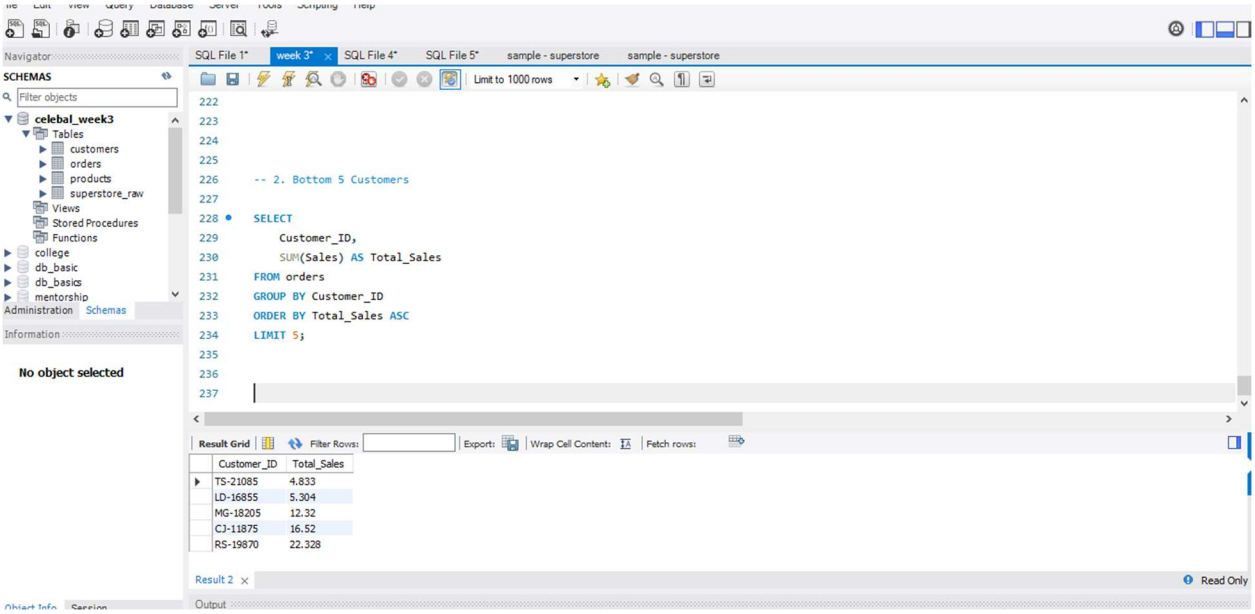
Insight: This query identifies the five customers with the lowest total sales, which helps in analyzing low-performing customer segments.

3.Customers Who Made Only One Order

Query:

```
SELECT
 Customer_ID,
    COUNT(Order_ID) AS Total_Orders
FROM orders
GROUP BY Customer_ID
HAVING COUNT(Order_ID) = 1
```

Output:



Insight:

This query identifies customers who placed only one order, which can help the business improve customer retention and encourage repeat purchases.

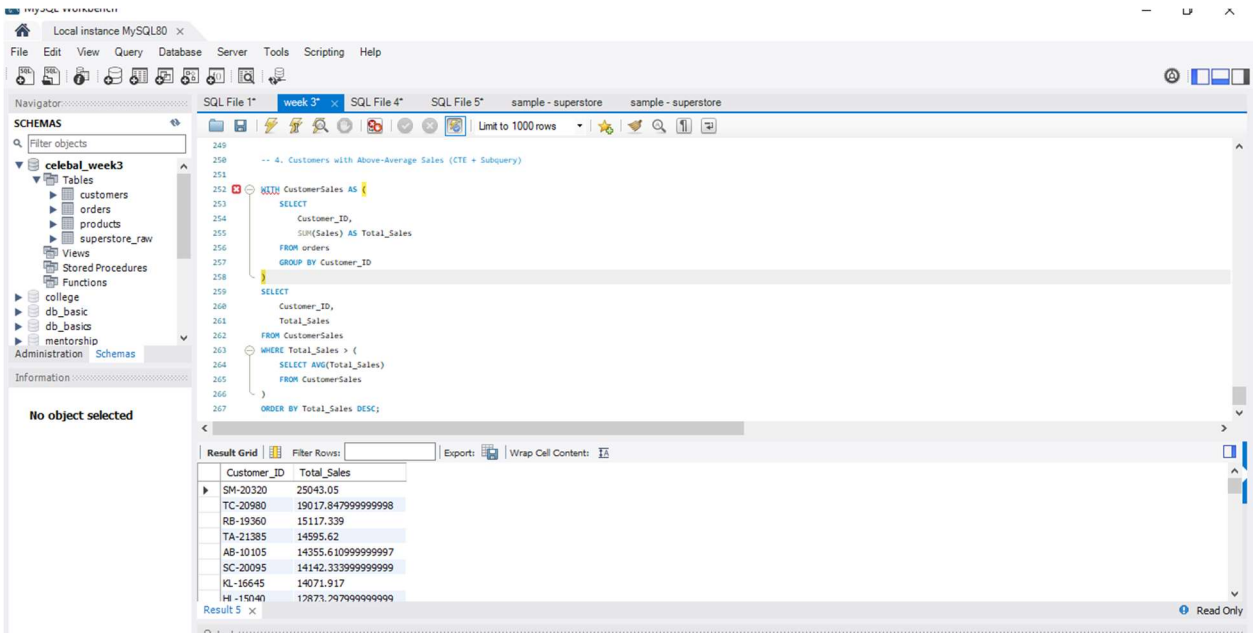
4.Customers with Above-Average Sales (CTE + Subquery)

Query:

```
WITH CustomerSales AS (
    SELECT
        Customer_ID,
        SUM(Sales) AS Total_Sales
    FROM orders
    GROUP BY Customer_ID
```

```
)
SELECT
    Customer_ID,
    Total_Sales
FROM CustomerSales
WHERE Total_Sales > (
    SELECT AVG(Total_Sales)
    FROM CustomerSales
)
ORDER BY Total_Sales DESC;
```

Output:



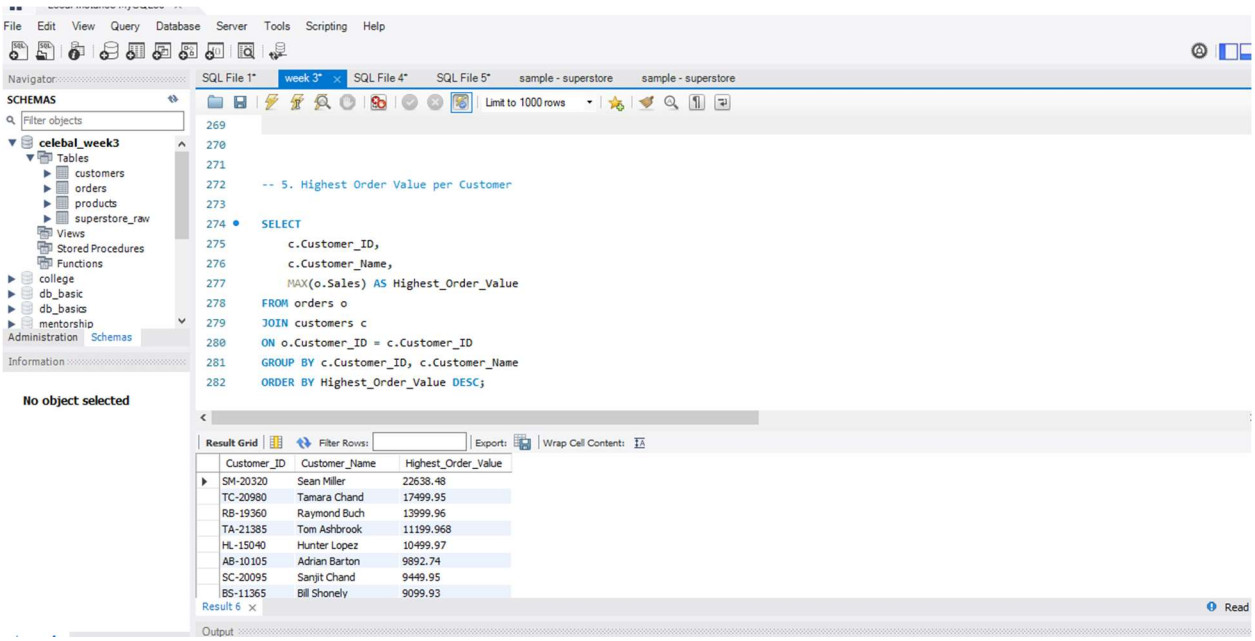
Insight: This query identifies customers whose total sales are greater than the average customer sales, highlighting high-performing customers who contribute significantly to revenue.

5. Highest Order Value per Customer

Query:

```
SELECT
    c.Customer_ID,
    c.Customer_Name,
    MAX(o.Sales) AS Highest_Order_Value
FROM orders o
JOIN customers c
ON o.Customer_ID = c.Customer_ID
GROUP BY c.Customer_ID, c.Customer_Name
ORDER BY Highest_Order_Value DESC;
```

Output:



Insight:

This query displays the highest order value for each customer, helping identify each customer's largest purchase and spending behavior

Conclusion

- Successfully applied **Subqueries** to filter and analyze sales data.
- Used **Common Table Expressions (CTEs)** to calculate customer-wise total sales.
- Implemented **Window Functions** (RANK() and ROW_NUMBER()) to rank customers based on sales performance.
- Combined **JOIN**, **CTE**, and **Window Functions** to generate a complete customer sales report.
- Identified top-performing customers, customers with low sales, and customers with single orders, enabling meaningful business analysis.